VIDYA: The AI Knowledge Person

Comprehensive Engineering Study Guide: System Design, Code Lifecycle, Project Execution & Troubleshooting Case Studies

---

1. System Design: Evolution & Architecture Blueprint

VIDYA started as a basic RAG pipeline and systematically evolved through 3 major architectural phases into a resilient, multimodal AI teaching agent.

Phase	Core Problem Solved	Technology Stack	Key Architectural Limitation
V1: Core RAG	Reading student textbooks and answering domain-specific queries without hallucinations.	pypdf, SentenceTransformers (MiniLM-L6), ChromaDB, Groq (Llama-3.3-70B)	Single sequential pipeline; rigid flow; no ability to choose alternative actions or do math.
V2: LangGraph & Tooling	Dynamic decision making: routing queries between PDF knowledge, web search, and calculation.	LangGraph StateGraph, DuckDuckGo Search (DDGS), Python AST Safe Calculator	Blind to visual elements (diagrams, flowcharts, architectures, and tables in PDFs).
V3: Visual Retrieval & VLM	Multimodal document tutoring: seeing actual page diagrams, figures, and structural layouts.	PyMuPDF (Fitz image rendering), Google GenAI SDK (Gemini 3.8 Flash Vision), Intent-gated routing	Cloud API rate limits (5 RPM on free tier); solved via smart intent filtering and backoff retries.

End-to-End System Architecture (LangGraph Flow):

2. Code Lifecycle: How Data Travels from Start to End

Understanding exactly where code starts, which file executes next, and how states are updated is critical for full-stack AI engineering.

Step & File	What Executes	State Transition (VIDYAState)
1. Entry Point app/main.py	Configures stdout encoding (UTF-8) on Windows. Triggers Ingest pipeline followed by QA test queries.	Initializes empty dictionary conforming to VIDYAState schema.

2. Ingest Graph app/graph.py app/nodes.py	Executes sequential ingestion: • load_pdf_node: Extracts text via pypdf. • split_text_node: Creates 100-word chunks (20-word overlap). • embed_node: Vectorizes chunks into 384-dim embeddings. • store_node: Persists vectors into ChromaDB. • visual_index_node: PyMuPDF renders pages to PNG in data/rendered_pages/. • suggest_node: Generates 3 student study questions.	Populates: pdf_text, chunks, embeddings, and suggestions.
3. Router Node app/nodes.py app/llm.py	router_node invokes classify_intent(query): • Rule 1: Checks for PDF constraint phrases ('from my pdf'). • Rule 2: Regex check for arithmetic/percentages ('17.5% of 85000'). • Rule 3: Keywords for real-time web ('latest', 'search online'). • Fallback: Zero-shot classification via Groq LLM.	Outputs route_choice: ('RAG', 'CALCULATOR', 'WEB_SEARCH', 'GENERAL').
4. Conditional Execution app/graph.py (Select Route)	LangGraph conditional edge branches the graph: • If CALCULATOR -> calculator_node (safe math evaluation). • If WEB_SEARCH -> web_search_node (DuckDuckGo web query). • If RAG -> retrieve_node (ChromaDB top-3 chunks + visual page reference).	Sets tool_used, tool_result, or context_chunks & visual_pages.
5. Answer Generation app/nodes.py app/vision/visual_retriever.py	generate_answer_node checks visual keywords: • **Visual Query** ('flowchart', 'diagram', 'chart'): Routes to Gemini 3.8 Flash Vision with page image. • **Standard Query**: Routes to Groq Llama-3.3-70B fast text tutor.	Produces final answer text formatted for the student.

3. How to Run AI Projects: Professional Engineering Standards

A common point of confusion for developers is how production projects are executed, managed, and debugged compared to single-file scripts.

• **1. Virtual Environment Isolation:** Never install packages globally into your operating system. Always use python -m venv venv and activate it (venv\Scripts\activate on Windows or source venv/bin/activate on Mac/Linux). This isolates conflicting dependencies.

• **2. Module Execution (python -m app.main vs python app/main.py):** When running an application that uses package-relative imports (like from app.nodes import ...), running python app/main.py fails with *ModuleNotFoundError: No module named 'app'* because Python sets sys.path to app/. Running python -m app.main from the project root correctly preserves the root directory on sys.path.

• **3. Environment Variable Security (.env):** API keys (GROQ_API_KEY, GEMINI_API_KEY) must never be hardcoded in Python files. Use python-dotenv to read .env at startup, and ensure .env is added to .gitignore.

• **4. Separation of Concerns (File Topology):**
• app/graph_state.py: The single source of truth data structure (TypedDict).

• app/nodes.py: Pure functions that consume State and return dict updates.
• app/graph.py: Graph assembly, edge wiring, and compilation.
• app/tools.py & app/vision/: Independent, testable domain modules.

4. Real-World Engineering Bugs & How We Overcame Them

During the development of VIDYA V3, we encountered 5 critical, production-grade blockers. Here is the exact problem breakdown, why it happened, and how we engineered the solution.

Issue & Root Cause	Why It Failed (Technical Breakdown)	Engineering Solution Implemented
1. ColPali Storage & Vector Store Blocker	ColPali/ColSmol requires downloading 5 to 8 GB PaliGemma model checkpoints locally. Furthermore, ColPali uses multi-vector late interaction (MaxSim), which ChromaDB cannot natively index without custom C++ engines or heavy Vespa/Qdrant clusters.	**Lightweight VLM Paradigm:** Decoupled text retrieval from vision. PyMuPDF renders pages to lightweight PNGs at ingest. At query time, only relevant pages are sent to Gemini Vision API. Zero local storage overhead, zero GPU requirement, instant execution.
2. Groq Vision Model Decommissioned Error 400: model_decommissioned	Groq retired llama-3.2-90b-vision-preview. When the fallback path attempted to call this model, the API rejected the request completely.	**Resilient Fallback:** Diverted Groq fallback directly to the stable llama-3.3-70b-versatile text LLM, ensuring that if vision fails, text RAG continues without crashing.
3. Gemini Deprecated Models & Free Tier Rate Limits Error 404 & 429 Quota Exceeded	• **404 Error:** Google deprecated gemini-1.5-flash and gemini-2.5-flash for new keys. • **429 Error:** Free tier accounts enforce a strict **5 Requests Per Minute (5 RPM)** ceiling. Running 6 test queries back-to-back exhausted the quota.	**Dual-Layer Defense:** 1. **Model Migration:** Upgraded to official gemini-3.8-flash. 2. **Smart Intent Gating:** Text questions use Groq; only queries with visual keywords ('diagram', 'chart') hit Gemini. 3. **Exponential Backoff:** Added auto-retry logic with 12s sleep on 429 errors.
4. Windows Console Unicode Encoding UnicodeEncodeError: charmap codec	Windows PowerShell uses legacy cp1252 code page by default. Printing Unicode arrow characters (→) caused catastrophic crashes inside console stdout.	**Safe Encoding Standard:** Replaced non-ASCII symbols with standard ASCII (->) across all graph nodes, and configured sys.stdout.reconfigure(encoding='utf-8') at startup.
5. Google GenAI SDK Migration & IDE Lint Warning	• Google sunsetted google.generativeai with a deprecation warning. • The new google-genai uses a dynamic namespace package that IDE type checkers (Pyright/Pylance) failed to resolve initially.	**Modern SDK Upgrade:** Migrated to from google import genai with types.Part.from_bytes(). Added graceful exception-guarded imports with # type: ignore to achieve 100% clean IDE analysis and zero console warnings.

5. Summary & Engineering Takeaways

✔ **Architectures Must Be Composable:** Decoupling LLMs behind abstract classes (like VisualRetriever) ensures you can switch between Gemini, ColPali, or Ollama without touching any LangGraph node logic.

✔ **Never Loop Blindly on Failures:** When an AI or distributed pipeline encounters unexpected status codes (400, 404, 429), stop execution, inspect the exact payload/trace, formulate hypotheses, and apply deliberate fixes instead of repetitive retries.

✔ **Multimodal Grounding Prevents Hallucination:** In Test 6, when asked if diagrams existed in a text-only PDF, text-only LLMs invented fake flowcharts. Gemini Vision examined the actual rendered page image and accurately stated: *'No diagrams exist in this document.'* Visual grounding is critical for truthfulness.

✔ **Rate Limits Demand Intent Filtering:** In production, API calls cost money and have latency. Filter intents upstream so only queries that strictly need expensive multimodal models trigger them.